

Wait / Block / Wakeup

条件不满足时，执行权怎样暂存并在事件后重新竞争 CPU

408 · OS-B01

Mother Interface

Condition Unsatisfied → Wait-Queue Enqueue → Blocked → Event/Condition Change → Wake → Runnable –

Bridge 只拥有“条件、等待队列与 task state”之间的控制权交接；同步原语、设备完成和调度策略仍由各自 Topic 拥有。

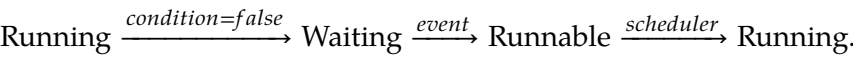
Owners 与边界

OS01 Owns task state、runnable 与调度；OS02 Owns lock/condition/semaphore 的同步条件；OS04 Owns I/O request/completion。本册 Owns 等待队列上的状态转换与重入检查，不 Own 调度算法或事件本体。

1 为什么必须先入队再阻塞

若线程先把自己标成 Blocked 再入队，事件可能在两步之间发生，唤醒者看不到等待者，形成 lost wakeup。安全协议是：在保护条件的同步范围内检查条件、登记等待者、改变 task state，再释放保护并让出执行权。唤醒后必须重新检查条件，因为多个等待者可能竞争同一资源。

2 最小状态机



“Wake” 只表示获得运行资格，不保证立即运行，也不保证条件仍为真；“Block” 也不等于进程退出。

3 最小例子与检查

生产者向空队列放入元素后唤醒消费者。消费者醒来先重新取得保护、检查队列非空，再取出元素；若队列仍为空则重新入队等待。题目出现 sleep/wakeup、I/O 等待或条件变量时，逐项追踪：谁在队列、task state 是什么、谁改变条件、谁触发 runnable、何时可能发生调度。

Anti-Bridge

wake ≠ “直接运行”；中断完成 ≠ 调度完成；一次偶然唤醒 ≠ 条件已经满足。任何检查条件的代码都不能把 wake 当作事实证明。

Compression

条件由谁改变？等待者何时登记？谁拥有唤醒资格？唤醒后为什么还要循环检查？